Project Status: Adaptive HTTP Download Manager

As of 29 Sep 2026. For the team — most of you last saw this at the RangeTracker stage, so here's everything since.

TL;DR

- The downloader is done and stable end to end: tracker, fetcher, assembler, worker pool, controllers, policy interface, RL adapter, controlled server, tc/netem, the whole thing.
- Headline result: on our D2 stress workload, a fixed config (16 connections, 4 MiB ranges) finishes **62% faster than aria2** — 30/30 runs, geometric ratio 0.376. It reproduces cleanly and it's frozen.
- The reason is specific: request granularity × request start timing × the server's per-response pacing. It is *not* "adaptation beats aria2" — the fixed config does it.
- We then spent a lot of effort trying to turn this into a genuine adaptive (joint N+S) result. Honest summary: on the D2 mechanism, and on every mechanism we screened, **within-download adaptation cannot beat the best fixed config**, and we now know exactly why.
- The original RL direction (LinUCB, DQN v1–v6) is evaluated and negative/descriptive. Documented, not abandoned.
- Current plan: package this as an IEEE-style paper — mechanism + 62% practical win + one gated adaptive extension — with about a month to go.

1. What exists now (everything after RangeTracker)

Layer	What it does	State
Fetcher	single HTTP range request, per-attempt metrics, retry plumbing	done, tested
Assembler	writes completed ranges at the right offsets	done
RangeTracker	half-open range lifecycle, pending/in-flight/completed	done (you've seen this)
Worker pool	dynamic N; workers retire between ranges; in-flight requests are never resized	done

Layer	What it does	State
Controllers	concurrency + chunk-size controllers with bounds and steps	done
Policies	fixed, throughput (N), chunk-size (S), joint (alternates N/S)	done
policyRunner	serializes decisions, computes reward/credit assignment	done
RL	LinUCB + adapter; DQN v1–v6 (Python training, Go inference)	done; results negative/descriptive
Controlled server	deterministic *logical* object (no 512 MiB allocations), per-response pacing, wall-clock rate schedules, fault injection	done
Network emulation	tc/netem wrapper (latency, loss, rate)	done
Experiment matrix + CLI	4 baseline modes + opt-in linucb, aggregates, control trajectories	done
aria2 harnesses	calibration, wall-clock study, competitive benchmark	done
Replay infra	mm-link Stage 0 replay, M3 mobile trace replay	done

2. Headline: 62.4% faster than aria2 (D2 workload)

D2 in one paragraph: 512 MiB file, one HTTP/1.1 server that paces each response at a rate sampled when the request starts and holds it — 640 KiB/s for the first 2 s, 160 KiB/s from 2–4 s, 2500 KiB/s after 4 s. Plus 40 ms RTT and 0.4% loss on loopback. It's a controlled stress test, not a real network.

Final 30-pair block:

	ours (16 conn, 4 MiB)	aria2 1.37 (split=16, 32 MiB pieces)
mean	20.12 s	53.56 s
median	20.02 s	53.32 s
geometric ratio	0.3756, CI [0.3728, 0.3785]	—

	ours (16 conn, 4 MiB)	aria2 1.37 (split=16, 32 MiB pieces)
wins	30/30, permutation p 5e-5	—

Why: across those 30 runs aria2 issued 450 requests (15 per run) of 32 MiB, and all of them start before the 2 s mark, so each is held at 640 KiB/s for ~51 s. We issued 3840 (128 per run) of 4 MiB; 480 start early, the other 3360 start after the recovery and get 2.5 MiB/s. Granularity × timing × pacing.

Controls that back this up:

- Granularity-matched control (ours, 16 conn, but 32 MiB ranges): ratio 0.862 [0.839, 0.885] — most of the win disappears once granularity matches.
- Constant-rate control (2.5 MB/s, no transition): ours-4M 15.3 s, ours-32M 16.7 s, aria2 24.8 s — the 4M-vs-32M gap collapses, so it's the phase transition doing the work.
- Fresh D2 reproduction with the old DQN v6 policy: 20/20, ratio 0.523 — the effect isn't a one-off.

We also ran the full aria2 split frontier (1/2/4/8/16 → medians 833/419/210/106/53 s) and a 40-config our-side frontier. split=16 and (16 conn, 4 MiB) are the strongest on each side — no cherry-picking.

3. The mechanism study: when can adaptation help?

We ran a series of screens (analytical first, live where it made sense) looking for a mechanism where the best (N,S) changes with the state, so a controller could beat a fixed config. Results:

Study	Mechanism	Outcome
Joint N+S phase-pacing (analytical)	server rate transition time varies	N=*16 always; S* moves 1M 4M; best causal controller loses to fixed (best +1.1%, optimistic bound +3.4%)
Live queueing (live)	shared FIFO bottleneck + background traffic	real queueing (RTT 0.04→86 ms, drops, retransmissions) but work-conserving; N*=16; gate failed

Study	Mechanism	Outcome
Correlated loss (analytical)	burst loss + range retries	16 MiB ranges fragile under bursts (33% attempt failure, 48% download aborts, +36% penalty), but N monotone in 9/9 variants; no joint opportunity
N-dependent server throttling (analytical)	the "promised" N-dependent throttling (sim_env_v5 semantics)	no qualifying scenario: high N hits the 3-attempt retry abort cliff, and the S* optimum sits at the timeout boundary where N degenerates. Stopped before live work
Adaptive-N screen (analytical)	server capacity degrading with N	the only model with an interior N optimum (+6.7% causal), but it never survived live validation

Takeaway: mechanisms that create a state-dependent N optimum are either not live-grounded or they collide with the downloader's retry/timeout semantics. The live-grounded mechanisms are work-conserving — more N is always more throughput.

4. Track 1: we did run a real joint adaptive benchmark on D2

Since the fixed win "isn't adaptive", we ran the existing adaptive policies on the same D2 workload: 20 blocks × 4 modes, randomized order, 80/80 valid runs.

mode	mean completion
fixed (16, 4 MiB)	20.25 s
N-only adaptive	36.93 s
S-only adaptive	64.97 s
joint adaptive N+S	46.91 s

- Joint vs fixed: ratio 2.32 (0/20 wins). Joint vs aria2: ratio 0.876 → 12.4% faster than aria2.

- The joint policy *did* adapt: 20/20 runs changed both N and S, ending at (6, 6 MiB).
- Why it's slow: decisions only happen when a request completes, so with 4 MiB ranges the first actionable decision lands at ~8.6 s — after the pacing recovery. By then the per-request throughput signal has saturated (~2.2 MB/s), the ±10% rule stops firing, and the controller freezes after 4 actions. It never ramps to 16 connections.
- Classification: BEATS_ARIA2_ONLY. Beating aria2 here is *not* adaptation value (the fixed config beats it by 62%).

Useful negative: the bottleneck is the policy signal/cadence, not the idea of adaptation.

5. Other campaigns, briefly

- **Five-trace oracle campaign** (the "does RL have headroom" question): Stage A and B ran clean (15/15 fixed-C replays, 88/88 tau runs), but the pre-registered final gate **failed** (LCB_95 = −0.0026 vs the +0.10 threshold). No RL/adaptive superiority claim.
- **Chunk-size salvage (v7)**: Stage 0S pilot passed (32/32) but the scaling analysis showed the frozen switch strategies have non-positive relative gain at every file size; the mechanism pivot found no robust state-dependent S optimum. No confirmatory run.
- **LinUCB/DQN**: LinUCB locks onto one arm; periodic exploration moves the posterior but rarely the preference. DQN candidates v2–v6 were mostly slower than aria2; v6 is the exception (48% faster than aria2) but still slower than our fixed config.

6. What's frozen (please don't touch)

Artifact	What it is
`testdata/aria2_competitive_benchmark/`	the 62.4% result + reproduction block
`testdata/diagnostics/long_duration_h2_diagnosis/`	h2 diagnosis evidence + mechanism writeup
`/home/vscode/final_fixed_c_stage_a`, `final_tau_stage_b`, `final_oracle_analysis`	five-trace campaign (gate FAILED)
`testdata/salvage_s_*`	chunk-size salvage: protocol v7, Stage 0S (GO), scaling, pivot
`testdata/adaptive_n_mechanism_screen/`, `testdata/final_project_synthesis/`	N screen + synthesis
`testdata/joint_ns_phase_pacing_screen/`, `live_joint_ns_queueing/`, `joint_ns_loss_mechanism/`	the mechanism screens
`testdata/joint_ns_d2/`	Track 1

Artifact	What it is
`testdata/n_dependent_server_joint_fit/`	Track 2 pre-screen (stopped)
`docs/PROJECT_CONTEXT.md`	living doc — if you read one thing, read this

7. Where we go from here (final month)

Goal: a positive, publishable paper. Realistic target: IEEE Access or a letters venue (ToN/TNSM would need real-world deployment evidence we can't produce in a month). The paper would be built around the mechanism + the 62% win, plus one adaptive extension:

Option	Claim	Expected size	Risk
A. Workload-dependent selection	a controller picks (N,S) per workload from a predeclared family; beats the best single fixed config by up to 7.9% (mean 3.1%) and aria2 by 50–62%	modest but real	low
B. Controlled N-dependent scenario (within-download)	implement the promised N-dependent throttling as a *soft* rate model (no abort cliff); if the live gate shows N* and S* both move, a causal joint controller beats the best fixed + aria2 within downloads	5–15% expected	medium; gated

Option	Claim	Expected size	Risk
Realism anchors	validate the granularity effect on mm-link replays of measured LTE/UMTS traces and/or a real Apache2 rate-limited server	—	low-medium
Baselines	add curl/wget/naive to the aria2 comparison	—	low

Rough plan: Week 1 pre-screen B + predeclare A + start the paper; Week 2 implement B (or pivot to A) + realism anchors; Week 3 adaptive runs + baseline sweep; Week 4 analysis + writing + buffer. B is gated: if the live fixed grid doesn't show both N* and S* moving, we stop B and go with A. No tuning to force it.

8. Open questions for the team

1. Venue: IEEE Access, or a letters venue? And is the 1-month deadline the submission itself or the course deadline (journal later)?
2. Adaptive claim: is B-or-A acceptable, or must the headline be within-download adaptation (making B mandatory)?
3. Realism anchor: mm-link traces, Apache2, both, or neither?
4. Defaults — shout if you disagree: B-first with A fallback; controlled-scenario framing; reuse the frozen evidence with explicit labels; baselines aria2+curl+wget+naive; keep the RL results as honest limits, no new RL work.

9. Where to look

- `docs/PROJECT_CONTEXT.md` — everything, in detail.
- `testdata/aria2_competitive_benchmark/reports/win_mechanism.md` — the 62% result.
- `testdata/joint_ns_d2/reports/joint_ns_d2_report.md` — Track 1.
- `testdata/n_dependent_server_joint_ns/reports/n_dependent_server_joint_ns_report.md` — Track 2 stop.
- `reports/` — latest report copies.